Synthetic Library Database Design and Implementation

Appendix:

Code for database creation and tables creation

```
import sqlite3
import random
from datetime import datetime, timedelta

# 1. Initialization and fixed seed for reproducibility
random.seed(20261106)
conn = sqlite3.connect('synthetic_library.db')
cursor = conn.cursor()

# 2. Database Design and Schema Creation
cursor.executescript("""
-- 10 rows: Catalogue classification
CREATE TABLE categories (
    category_id INTEGER PRIMARY KEY,
    category_name TEXT NOT NULL,
    display_priority INTEGER CHECK(display_priority > 0)
);

-- 500 rows: Synthetic member profiles
CREATE TABLE members (
    member_id INTEGER PRIMARY KEY,
    display_name TEXT NOT NULL,
    synthetic_email TEXT NOT NULL UNIQUE,
    region TEXT CHECK(region IN ('North', 'South', 'East', 'West', 'Central')),
    joined_date TEXT NOT NULL,
    loyalty_tier TEXT,
    satisfaction_level TEXT CHECK(satisfaction_level IN ('Low', 'Medium',
'High', 'Very High'))
);

-- 150 rows: Books with stock, cost, and interval-scaled year data
CREATE TABLE books (
    book_id INTEGER PRIMARY KEY,
    category_id INTEGER REFERENCES categories(category_id),
    title TEXT NOT NULL,
    unit_fee_cents INTEGER CHECK(unit_fee_cents >= 0),
    available_stock INTEGER CHECK(available_stock >= 0),
    publication_year REAL,
    popularity_rank INTEGER
);

-- 1,500 rows: Borrowing records linked to members
CREATE TABLE loans (
    loan_id INTEGER PRIMARY KEY,
    member_id INTEGER REFERENCES members(member_id),
```

```
    loan_date TEXT NOT NULL,
    loan_status TEXT CHECK(loan_status IN ('Active', 'Completed', 'Cancelled')),
    branch_region TEXT,
    processing_cents INTEGER CHECK(processing_cents >= 0),
    loan_total_cents INTEGER CHECK(loan_total_cents >= 0)
);

-- ~3,696 rows: Individual books in a loan, featuring a composite primary key
CREATE TABLE loan_items (
    loan_id INTEGER REFERENCES loans(loan_id),
    line_number INTEGER,
    book_id INTEGER REFERENCES books(book_id),
    quantity INTEGER CHECK(quantity BETWEEN 1 AND 5),
    unit_fee_cents INTEGER CHECK(unit_fee_cents >= 0),
    PRIMARY KEY (loan_id, line_number)
);

-- 1,500 rows: 1:1 relationship with loans
CREATE TABLE payments (
    payment_id INTEGER PRIMARY KEY,
    loan_id INTEGER UNIQUE REFERENCES loans(loan_id),
    method TEXT,
    payment_status TEXT CHECK(payment_status IN ('Paid', 'Pending', 'Void')),
    amount_cents INTEGER CHECK(amount_cents >= 0),
    paid_date TEXT
);
""")

# 3. Synthetic Data Generation

# Generate Categories
categories = ['Fiction', 'Non-Fiction', 'Sci-Fi', 'Biography', 'History',
'Science', 'Children', 'Mystery', 'Romance', 'Poetry']
for i, cat in enumerate(categories, 1):
    cursor.execute("INSERT INTO categories VALUES (?, ?, ?)", (i, cat, i))

# Generate Members
regions = ['North', 'South', 'East', 'West', 'Central']
satisfaction = ['Low', 'Medium', 'High', 'Very High']
tiers = ['Standard', 'Premium', 'Elite']
start_date = datetime(2020, 1, 1)

for i in range(1, 501):
    join_date = start_date + timedelta(days=random.randint(0, 1500))
    email = f"member{i}@example.com"
    cursor.execute("""
        INSERT INTO members (member_id, display_name, synthetic_email, region,
joined_date, loyalty_tier, satisfaction_level)
        VALUES (?, ?, ?, ?, ?, ?, ?)
    """, (i, f"Member {i}", email, random.choice(regions),
join_date.strftime('%Y-%m-%d'), random.choice(tiers),
random.choice(satisfaction)))

# Generate Books
for i in range(1, 151):
```

```
    cursor.execute("""
        INSERT INTO books (book_id, category_id, title, unit_fee_cents,
available_stock, publication_year, popularity_rank)
        VALUES (?, ?, ?, ?, ?, ?, ?)
    """, (i, random.randint(1, 10), f"Book Title {i}", random.randint(50, 500),
random.randint(0, 50), random.randint(1900, 2026), random.randint(1, 100)))

# Generate Loans, Loan Items, and Payments
loan_items_count = 0
for loan_id in range(1, 1501):
    member_id = random.randint(1, 500)
    cursor.execute("SELECT joined_date FROM members WHERE member_id = ?",
(member_id,))
    member_join_date = datetime.strptime(cursor.fetchone()[0], '%Y-%m-%d')
    loan_date = member_join_date + timedelta(days=random.randint(1, 500))

    status_choices = ['Active', 'Completed', 'Cancelled']
    loan_status = random.choices(status_choices, weights=[20, 70, 10])[0]

    # Generate 1 to 4 items per loan
    num_items = random.randint(1, 4)
    subtotal_cents = 0

    for line_num in range(1, num_items + 1):
        book_id = random.randint(1, 150)
        cursor.execute("SELECT unit_fee_cents FROM books WHERE book_id = ?",
(book_id,))
        unit_fee = cursor.fetchone()[0]
        qty = random.randint(1, 5)

        if loan_status != 'Cancelled':
            subtotal_cents += unit_fee * qty

        cursor.execute("""
            INSERT INTO loan_items (loan_id, line_number, book_id, quantity,
unit_fee_cents)
            VALUES (?, ?, ?, ?, ?)
        """, (loan_id, line_num, book_id, qty, unit_fee))
        loan_items_count += 1

        # Cap at requirement of ~3696 rows
        if loan_items_count >= 3696:
            break

    processing_cents = random.randint(100, 500) if loan_status != 'Cancelled'
else 0
    total_due = subtotal_cents + processing_cents if loan_status != 'Cancelled'
else 0

    cursor.execute("""
        INSERT INTO loans (loan_id, member_id, loan_date, loan_status,
branch_region, processing_cents, loan_total_cents)
        VALUES (?, ?, ?, ?, ?, ?, ?)
    """, (loan_id, member_id, loan_date.strftime('%Y-%m-%d'), loan_status,
random.choice(regions), processing_cents, total_due))
```

```
    # Generate Payment tied to loan status
    payment_status = 'Paid' if loan_status == 'Completed' else 'Void' if
loan_status == 'Cancelled' else 'Pending'
    paid_date = (loan_date + timedelta(days=random.randint(1,
30))).strftime('%Y-%m-%d') if payment_status == 'Paid' else None

    cursor.execute("""
        INSERT INTO payments (payment_id, loan_id, method, payment_status,
amount_cents, paid_date)
        VALUES (?, ?, ?, ?, ?, ?)
    """, (loan_id, loan_id, random.choice(['Credit', 'Debit', 'Cash']),
payment_status, total_due, paid_date))

conn.commit()

# 4. Testing and Validation Checks
cursor.execute("PRAGMA integrity_check;")
print("Integrity Check:", cursor.fetchone()[0])
cursor.execute("PRAGMA foreign_key_check;")
print("Foreign Key Violations:", cursor.fetchall())

conn.close()
```

Code to view tables

```
import sqlite3

conn = sqlite3.connect('synthetic_library.db')
cursor = conn.cursor()

tables = ['categories', 'members', 'books', 'loans', 'loan_items', 'payments']

for table in tables:
    cursor.execute(f"SELECT * FROM {table} LIMIT 5;")
    rows = cursor.fetchall()

    # Fetch column names for context
    column_names = [description[0] for description in cursor.description]

    print(f"Top 5 rows in table: {table.upper()}")
    print(" | ".join(column_names))
    print("-" * 50)
    for row in rows:
        print(row)
    print("\n")

conn.close()
```

Code for validation

```
import sqlite3

conn = sqlite3.connect('synthetic_library.db')
cursor = conn.cursor()
```

```
# 1. Database and Referential Integrity
# PRAGMA checks to confirm internal consistency and missing foreign keys
cursor.execute("PRAGMA integrity_check;")
print(f"Integrity Check: {cursor.fetchone()[0]}")

cursor.execute("PRAGMA foreign_key_check;")
fk_violations = cursor.fetchall()
print(f"Foreign Key Violations (should be empty): {fk_violations}")

# 2. Constraint Enforcement
# Attempting to insert an invalid negative quantity and unknown loan ID[cite: 1]
print("\nTesting Constraint Enforcement:")
try:
    cursor.execute("INSERT INTO loan_items (loan_id, line_number, book_id,
quantity, unit_fee_cents) VALUES (9999, 1, 1, -5, 100)")
except sqlite3.IntegrityError as e:
    print(f"Constraint successfully caught invalid entry: {e}")

# 3. Detection of Duplicate Primary Keys
# Checking for duplicate primary keys in the members table[cite: 1]
cursor.execute("""
    SELECT member_id, COUNT(*)
    FROM members
    GROUP BY member_id
    HAVING COUNT(*) > 1;
""")
print(f"\nDuplicate Member IDs (should be empty): {cursor.fetchall()}")

# 4. Business Logic Validation
# Verifying loan totals and payment amounts match status-specific rules[cite: 1]
cursor.execute("""
    SELECT l.loan_status, p.payment_status, COUNT(*), SUM(l.loan_total_cents),
SUM(p.amount_cents)
    FROM loans l
    JOIN payments p ON l.loan_id = p.loan_id
    GROUP BY l.loan_status, p.payment_status;
""")
print("\nBusiness Logic (Loan Status | Payment Status | Count | Loan Total Cents
| Payment Total Cents):")
for row in cursor.fetchall():
    print(row)

# 5. Aggregate Queries: Transaction Patterns & Product Quantities
# Analyzing transaction patterns by branch region[cite: 1]
cursor.execute("""
    SELECT branch_region, COUNT(loan_id), SUM(loan_total_cents) / 100.0 AS
revenue_dollars
    FROM loans
    WHERE loan_status = 'Completed'
    GROUP BY branch_region
    ORDER BY revenue_dollars DESC;
""")
print("\nTransaction Patterns by Region (Completed Loans):")
for row in cursor.fetchall():
```

```
    print(row)

# Analyzing book quantities[cite: 1]
cursor.execute("""
    SELECT b.title, SUM(li.quantity) AS total_borrowed
    FROM loan_items li
    JOIN books b ON li.book_id = b.book_id
    GROUP BY b.title
    ORDER BY total_borrowed DESC
    LIMIT 5;
""")
print("\nTop 5 Most Borrowed Books (Quantities):")
for row in cursor.fetchall():
    print(row)

# Analyzing payment information[cite: 1]
cursor.execute("""
    SELECT method, payment_status, COUNT(*), SUM(amount_cents) / 100.0 AS
total_dollars
    FROM payments
    GROUP BY method, payment_status
    ORDER BY method, payment_status;
""")
print("\nPayment Information Summary (Method | Status | Count | Total
Dollars):")
for row in cursor.fetchall():
    print(row)

conn.close()
```